

Snake játék és mesterséges intelligencia – integrált szakdolgozati dokumentáció

Ez a dokumentum a **megvalósított rendszer** részletes leírását adja szakdolgozati formában: architektúra, frontend és backend szerkezet, kódrészletek, AI modul, benchmark és eredmények. A hivatalos, követelményszintű specifikáció: [docs/György_Ákos_Szakdolgozat_Specifikáció.md](#). Terv-jellegű, nem implementált dokumentumok nem részei ennek az összefoglalónak.

Tartalomjegyzék

1. Absztrakt
 2. Bevezetés
 3. Rendszer áttekintése, monorepo és fejlesztői futtatás
 4. Technológiai stack és adatfolyam
 5. Frontend – technológiák, játéklógika, felület és téma
 6. Backend – szerkezet, adatbázis, auth és API
 7. AI szolgáltatás – FastAPI, stratégiák, példák
 8. Tanuló környezet és algoritmusok
 9. Benchmark: protokoll és eszközök
 10. Mérési eredmények és értelmezés
 11. Webes alkalmazás: funkionalitások (képernyők, auth, játék)
 12. Korlátok és következtetések
 13. Architektúra és döntések (jegyzetek)
 14. Irodalomjegyzék
-

1. Absztrakt

A szakdolgozat egy **Snake** játékot és több **mesterséges intelligencia alapú irányítási** megközelítést integráló **monorepo** alkalmazást mutat be. A rendszer három fő komponensből áll: **React + TypeScript** frontend (Canvas alapú játéktér), **Node.js + Express + SQLite** backend (felhasználók, JWT alapú bejelentkezés, pontszámok), valamint **Python FastAPI** AI szolgáltatás **WebSocketen** keresztül valós idejű döntéshozattal. A MI réteg **heurisztikus stratégiákat** (A^* , BFS, Hamilton-jellegű bejárások, előretekintés, minimax, biztonságvezérelt módszerek) és **tanuló** megoldásokat (**DQN**, **PPO**, **NEAT**) egyesít egy közös interfészen. Az összehasonlíthatóság érdekében **egységes benchmark** futtató méri az átlagos és medián pontszámot, a lépésszámot, a halálok megoszlását és az első étel elérésének arányát; az eredmények **JSON** formában tárolódnak, és a frontend **Statisztika** nézetén (§5.9), valamint **generált grafikonokon** is megjeleníthetők. A mérések egy konkrét futássorozat alapján azt mutatják, hogy a vizsgált **heurisztikák** (különösen az 1 lépéses előretekintés) ebben a környezetben **jelentősen felülmúlják** a dokumentált **tanuló** modelleket; utóbbiak teljesítménye erősen függ a jutalomfüggvényről, a tanítási időtől és a megfigyelés reprezentációjától.

Kulcsszavak: Snake játék, útkeresés, erősítéses tanulás, DQN, PPO, neuroevolúció, NEAT, FastAPI, React, benchmark.

2. Bevezetés

A Snake klasszikus, rácsalapú játék: a kígyó ételt gyűjt, nő, és elkerüli a falat és önmaga ütközését. Egyszerű szabályai mellett jól vizsgálhatók **determinisztikus** (szabály- és keresésalapú) és **tanuló** stratégiák. A projekt célja egy **reprodukálható** keretrendszer, amelyben ugyanazon pályán és protokoll mellett mérhető a teljesítmény, és amely **weben** is játszható, felhasználói fiókkal és eredménytárolással.

A bevezető motiváció összhangban áll a [docs/György_Ákos_Szakdolgozat_Specifikáció.md](#) általános célkitűzésével; a **tényleges megvalósítás** a specifikáció korai „tervezett” technológiai listájától eltér: a játéktér **natív Canvas** és **TypeScript játéklógika**, nem Phaser/p5.js.

3. Rendszer áttekintése, monorepo és fejlesztői futtatás

3.1 Fő eredmények (megvalósítva)

- Játszható Snake **böngészőben, játékos** és **MI** módban.
- **Többféle** MI stratégia egy **közös API** mögött (állapotból következő irány).
- **Regisztráció / bejelentkezés, profil, pontszámok** tárolása SQLite-ban.
- **Egységes benchmark** több stratégiára; **statisztika** oldal; **PNG grafikonok**.
- **Világos / sötét téma**, konfigurálható rácsméret és játékebesség.

3.2 Monorepo felépítés

Mappa / fájl	Tartalom
frontend/	React + TypeScript + Vite, játéklógika, UI, auth kliens
backend/	Node.js + Express + SQLite, auth, profil, pontszámok
ai_service/	Python FastAPI, WebSocket, stratégiák, SnakeEnv , tanítás
benchmarks/	run_strategy_benchmark.py , generate_benchmark_plots.py , results/ , plots/
docs/	Specifikáció, szakdolgozati dokumentáció, ai_docs/
Gyökér package.json	npm run dev , npm run dev:backend , npm run dev:ai , npm run dev:all
README.md (gyökér)	Követelmények, telepítés, futtatás, portok

3.3 Egyszerre mindhárom szolgáltatás ([dev:all](#))

A gyökérből [npm run dev:all](#) párhuzamosan indítja a frontendet (tipikusan [http://localhost:5173](#)), a backendet ([http://localhost:3000](#)) és az AI szolgáltatást ([http://localhost:8000](#)). A gyökérben a **concurrently** (devDependency) futtatja a három dev scriptet; a konzolban a logok színezett prefixekkel ([\[frontend\]](#), [\[backend\]](#), [\[ai\]](#)) különülnek el. Az [ai_service](#) mappában egy **package.json** biztosítja, hogy [npm run dev --prefix ai_service](#) (vagy [npm run dev:ai](#)) ugyanúgy indítsa a szervert: [python -m uvicorn src.main:app --reload --host 0.0.0.0 --port 8000](#).

4. Technológiai stack és adatfolyam

4.1 MI mód adatfolyama

1. A frontend tick-enként előállítja a játék pillanatképét (`getSnapshot`).
2. **WebSocket** üzenetben elküldi az `ai_service/ws` végpontra (opcionális `strategy` mező a kiválasztott azonosítóval).
3. A szerver a stratégiával kiszámolja a következő **irányt** (`Up / Right / Down / Left`).
4. A kliens alkalmazza az irányt, és frissíti a Canvas megjelenítést.

Ha a WebSocket nem elérhető, a kliens **helyi fallback** stratégiát használ (`frontend/src/ai/Strategy.ts` – egyszerűsített étel felé logika).

4.2 REST összefoglaló

- **Backend:** `POST /api/auth/register`, `POST /api/auth/login`, `GET/PATCH /api/profile/me`, `GET/POST /api/scores` – részletek: [6. Backend](#).
- **AI szolgáltatás:** `GET /health`, `GET /strategies`, `POST /next`, WebSocket `/ws`, valamint `GET /benchmark/summaries`, `GET /benchmark/plots/{filename}` a statisztika oldalhoz.

5. Frontend – technológiák, játéklógika, felület és téma

5.1 Technológiák

Vite 5, React 18, TypeScript 5.6, HTML5 Canvas a játéktérhez. A játéklógika **tiszta TypeScript** modulokban van (független a Reacttól), így tesztelhető és ugyanaz a pillanatkép-formátum használható az MI réteghez.

5.2 Fájlstruktúra (fontosabb részek)

```
frontend/
├─ index.html
├─ package.json, vite.config.ts, tsconfig.json
├─ public/
└─ src/
    ├─ main.tsx                # ThemeProvider, AuthProvider
    ├─ App.tsx                 # Képernyőváltás, játékciklus, auth
    ├─ theme.css               # Globális és téma CSS változók
    ├─ api.ts                  # REST (auth, profil, scores); benchmark URL-ek
    ├─ ThemeContext.tsx
    ├─ AuthContext.tsx
    ├─ core/                   # Játéklógika
    │   ├─ types.ts
    │   ├─ rng.ts
    │   ├─ board.ts, snake.ts, food.ts, collision.ts, score.ts
    │   ├─ game.ts             # Állapotgép, tick, getSnapshot
    │   └─ index.ts
    └─ ai/
        ├─ Strategy.ts
        ├─ strategies.ts
        └─ strategyBenchmarkDetail.ts
```

```

├── io/
│   ├── config.ts
│   └── storage.ts
├── hooks/
│   ├── useGameLoop.ts
│   ├── useAIGameLoop.ts
│   └── useAIWebSocket.ts
├── view/
│   ├── GameCanvas.tsx
│   └── HUD.tsx
└── ui/
    ├── Header.tsx, MainMenu.tsx, Settings.tsx
    ├── Results.tsx, Statistics.tsx
    ├── LoginForm.tsx, RegisterForm.tsx, Profile.tsx
    └── PlayerSettings.tsx, AISettings.tsx

```

5.3 Adatszerkezetek és stratégia interfész

A `core/types.ts` definiálja a `GameStateSnapshot` mezőit (snake, direction, food, rows, cols, seed, tick, score), a `GamePhase` értékeit és a `Strategy` interfészt. A `game.ts` kezeli a tick-et: irány (ellentétes tiltva), mozgás, ütközés, étel, `getSnapshot` az MI számára.

```

/** Játék állapotgép. */
export type GamePhase = 'INIT' | 'RUNNING' | 'PAUSED' | 'GAME_OVER'

/** Stratégia interfész (helyi / specifikáció szerinti elv). */
export interface Strategy {
  nextMove(state: GameStateSnapshot): Action
}

```

5.4 Példa: játék létrehozása és irányváltás

```

export function createGame(config: Partial<GameConfig> | null): GameState {
  const rows = config?.grid?.rows ?? DEFAULT_ROWS
  const cols = config?.grid?.cols ?? DEFAULT_COLS
  const tickMs = config?.tick_ms ?? DEFAULT_TICK_MS
  const seed = config?.seed ?? Date.now()
  setSeed(seed)
  const { body, direction } = createSnake(rows, cols)
  const food = placeFoodSeeded(rows, cols, body, next)
  return { phase: 'INIT', rows, cols, snakeBody: body, direction, food, score:
0, tick: 0, seed, tickMs }
}

export function setDirection(state: GameState, newDir: Direction): GameState {
  if (state.phase !== 'RUNNING' && state.phase !== 'INIT') return state
  if (isOpposite(state.direction, newDir)) return state
}

```

```
return { ...state, direction: newDir }
}
```

5.5 Perzisztencia (I/O)

A **konfiguráció** (rács, `tick_ms`, `seed`, `ai.strategy`) és az **eredmények** (pont, tick, hossz, mód, `aiStrategy`) a böngésző **localStorage**-ában tárolódnak (`io/config.ts`, `io/storage.ts`). Bejelentkezés után a játék végi pontszám `submitScore` hívással a backendre is kerül (`api.ts`).

5.6 Megjelenítés és UI

- **GameCanvas:** Canvas 2D; rács, kígyó (fej kiemelve), étel – színek a téma szerint (a Canvas nem használ CSS változókat közvetlenül, ezért a komponensben paletta).
- **HUD:** pont, hossz, lépés, sebesség (tick/s), státusz (Készül / Fut / Szünet / Vége); MI módban „MI: backend (stratégia)” vagy „MI: helyi”.
- **Beállítások:** két fül – **Játékos** és **MI**. Pálya **10–40** sor/oszlop, **tick 50–500** ms, opcionális seed. MI fülön **legördülő** stratégia lista (`AI_STRATEGIES`), rövid leírás a választás alatt.
- **App.tsx képernyők:** `menu`, `settings`, `results`, `statistics`, `game`, `login`, `register`, `profile`. Billentyűzet játékban: **nyilak** / **WASD**, **P** szünet, **R** új játék, **Enter** indítás INIT-ből.

5.7 Light / dark mód

A téma `[data-theme="dark"]` / `[data-theme="light"]` CSS változókkal van definiálva (`theme.css`); a `ThemeContext.tsx` / `ThemeProvider` beállítja a `document.documentElement` `data-theme` attribútumát és a **localStorage** kulcsot (`snake_theme`). A **Header** tartalmazza a téma váltó gombot.

```
[data-theme="dark"] {
  --bg: #0f1419;
  --surface: #2d3748;
  --text: #e2e8f0;
  --accent: #48bb78;
  /* ... további változók a theme.css-ben */
}
[data-theme="light"] {
  --bg: #f7fafc;
  --surface: #ffffff;
  --text: #2d3748;
  /* ... */
}
```

```
export function ThemeProvider({ children }: { children: ReactNode }) {
  const [theme, setThemeState] = useState<Theme>(getStoredTheme)
  useEffect(() => {
    document.documentElement.setAttribute('data-theme', theme)
    localStorage.setItem(STORAGE_KEY, theme)
  }, [theme])
  const toggleTheme = useCallback(() => setThemeState((prev) => (prev ===
```

```
'dark' ? 'light' : 'dark'))), [])
  return <ThemeContext.Provider value={{ theme, setTheme, toggleTheme }}>
{children}</ThemeContext.Provider>
}
```

5.8 Auth kontextus (kliens)

Az **AuthContext** tárolja a **JWT**-t és a **user** objektumot; **setAuth**, **logout**, **loadUser**. A token és a user JSON a **localStorage**-ban (**snake_token**, **snake_user**). A védett API hívások **Authorization: Bearer** fejléccet kapnak.

5.9 Statisztika oldal és adatvizualizáció

A **Statisztika** a főmenüből nyílik (**screen === 'statistics'**, komponens: **Statistics.tsx**). Ugyanúgy a frontend része, mint az eredmények vagy a beállítások; adatai az **AI szolgáltatás** benchmark kiszolgáló végpontjairól érkeznek, nem a Node backendről.

- **Elérés:** Főmenü → **Statisztika** (**App.tsx** → **Statistics**).
- **HTTP bázis:** **VITE_AI_HTTP_URL** vagy **http://localhost:8000** (**api.ts**).
- **Összefoglalók:** **GET /benchmark/summaries** – a szerver a repó **benchmarks/results/** mappájából gyűjti a **strategy_benchmark_*.json** fájlok metaadatait (átlag/medián pont és lépés, futások száma, halál okok, első étel arány stb.).
- **Grafikonok:** **GET /benchmark/plots/{filename}** – a **benchmarks/results/plots/** alatti előre generált **PNG** fájlok (pl. **generate_benchmark_plots.py** kimenete).
- **Táblázat:** soronként egy stratégia / egy benchmark futás összegzése; hiányzó halál-kulcsok **0%**-ként jelennek meg.
- **Szűrés:** „Mind”, „Egyik sem”, **Csak heurisztikák**, **Csak neurális hálók** (azonosítók: **dqn**, **ppo**, **neuroevolution** – összhangban a **STRATEGIES** regiszterrel és a frontend **strategies.ts** kategorizálásával).
- **Rendezés:** oszlopfejlécekre kattintva.
- **Részletek panel:** sorra kattintva: a **strategyBenchmarkDetail.ts** szövegei és az adott sor metrikái (**AI_STRATEGIES** / **strategies.ts** megjelenített nevek).
- **Lightbox:** a beágyazott PNG-kre kattintva nagyított megjelenítés.

Dokumentáció-összhang: a metrikaértelmezés és a kiértékelés írásos háttere:

docs/ai_docs/benchmark_eredmenyek_reszletes_kiertekeles.md,

docs/ai_docs/benchmark_kiertekeles_es_adatvizualizacio_utmutato.md, vázlat:

docs/ai_docs/szakdolgozat_statisztika_es_elemzes_struktura.md. A statisztika **nem** játszik szerepet a játék tickjében; csak olvasási / elemzési felület.

6. Backend – szerkezet, adatbázis, auth és API

6.1 Technológiák

Node.js, **Express 4**, **TypeScript** (tsx / tsc), **better-sqlite3**, **bcrypt** (jelszó hash, tipikusan 10 kör), **jsonwebtoken** (JWT, pl. 7 nap), **cors**. Alapértelmezett URL: **http://localhost:3000** (**VITE_API_URL** felülírhatja).

6.2 Fájlstruktúra

```

backend/
├── package.json, tsconfig.json
├── README.md
├── data/                # SQLite: snake.db (gitignore)
├── src/
│   ├── index.ts         # Express app, CORS, route-ok
│   ├── db/sqlite.ts     # getDb(), initSchema, migrációk
│   ├── middleware/auth.ts
│   └── routes/
│       ├── auth.ts
│       ├── profile.ts
│       └── scores.ts

```

6.3 Adatbázis és séma (összefoglaló)

SQLite fájl: `backend/data/snake.db`. Táblák: **users** (id, email UNIQUE, username UNIQUE, password_hash, created_at), **scores** (user_id FK, score, tick, length, mode **player|ai**, **ai_strategy** TEXT NULL – tetszőleges stratégia azonosító, **schema_version** migrációval a korlátozó CHECK eltávolítva a bővíthetőség miatt). Indexek: `scores(user_id)`, `scores(created_at DESC)`.

```

// Séma vázlat (sqlite.ts - initSchema)
CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  email TEXT UNIQUE NOT NULL,
  username TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  created_at TEXT NOT NULL DEFAULT (datetime('now'))
);
CREATE TABLE IF NOT EXISTS scores (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL REFERENCES users(id),
  score INTEGER NOT NULL,
  tick INTEGER NOT NULL,
  length INTEGER NOT NULL,
  mode TEXT NOT NULL CHECK (mode IN ('player', 'ai')),
  ai_strategy TEXT NULL,
  created_at TEXT NOT NULL DEFAULT (datetime('now'))
);

```

6.4 JWT middleware

```

export function authMiddleware(req: Request, res: Response, next:
NextFunction): void {
  const authHeader = req.headers.authorization
  if (!authHeader?.startsWith('Bearer ')) {
    res.status(401).json({ error: 'Hiányzó vagy érvénytelen token' })
    return
  }
  next()
}

```

```

    }
    const token = authHeader.slice(7)
    try {
      const payload = jwt.verify(token, JWT_SECRET) as JwtPayload
      ;(req as Request & { user: JwtPayload }).user = payload
      next()
    } catch {
      res.status(401).json({ error: 'Érvénytelen vagy lejárt token' })
    }
  }
}

```

6.5 Végpontok és üzleti szabályok

Módszer	Útvonal	Leírás
POST	/api/auth/register	email, username, password, passwordConfirm – validáció, bcrypt hash
POST	/api/auth/login	usernameOrEmail, password → { token, user }
GET	/api/profile/me	Bearer – felhasználó adatok
PATCH	/api/profile/me	Bearer – username módosítás
PATCH	/api/profile/me/password	Bearer – currentPassword, newPassword, newPasswordConfirm
GET	/api/scores	Bearer – saját eredmények listája
POST	/api/scores	Bearer – score, tick, length, mode, ai_strategy

Regisztráció: email formátum, felhasználónév hossz/karakterkészlet, jelszó minimum hossz, egyező megerősítés. **Éles** környezetben kötelező a **JWT_SECRET** beállítása.

7. AI szolgáltatás – FastAPI, stratégiák, példák

7.1 Technológiák és futtatás

Python 3.10+, FastAPI, uvicorn. A heurisztikus stratégiák többsége **csak standard könyvtárra** épít; a **DQN** tanítás és inferencia **PyTorch**-ot, a **PPO Stable-Baselines3**-at, a **NEAT** a **neat-python** csomagot használja, ha telepítve vannak (**requirements.txt**).

```

cd ai_service
pip install -r requirements.txt
python -m uvicorn src.main:app --reload --host 0.0.0.0 --port 8000

```

7.2 Közös interfész és állapot

Minden stratégia a **Strategy** absztrakt osztályból indul (`strategies/base.py`): egyetlen metódus, `next_move(state: GameState) -> Direction`.

A frontend **GameStateSnapshot** JSON a `parse_state` (`state.py`) révén **GameState**-té alakul (fej, kígyó teste, étel, rács, irány, tick, score). A `simulate_step(state, direction)` egy lépést szimulál ütközés nélkül vagy **None**-al jelez halált – ezt használják az előretekintő és minimax stratégiák.

Regiszter: `strategies/__init__.py` → **STRATEGIES** szótár: kulcs = azonosító (pl. `astar`), érték = stratégia osztály vagy gyárfüggvény. A WebSocket `/ws` és a **POST** `/next` üzenet opcionális **strategy** mezője ezt a kulcsot választja. A frontend **AI_STRATEGIES** listája ugyanezekhez igazított címkeket és leírásokat mutat.

Környezeti változók (példa): `AI_STRATEGY` (alapértelmezett stratégia), `DQN_MODEL_DIR`, illetve az **A*** opcionális **safety** kapcsolója konstruktorban / env-ben.

7.3 Fájlstruktúra (összefoglaló)

```
ai_service/
├── requirements.txt, README.md, models/      # .pt, .zip, .pkl - gitignore
│      szerint
├── training/                                # train_dqn.py, train_ppo.py, train_neat.py, ...
└── src/
    ├── main.py                             # FastAPI, /ws, /next, /strategies, /benchmark/...,
    CORS
    ├── state.py                             # GameState, parse_state, simulate_step, DELTA
    ├── env/snake_env.py                     # SnakeEnv, state_to_observation (tanítás +
    inferencia)
    └── strategies/
        ├── __init__.py                     # STRATEGIES
        ├── astar.py, bfs.py, greedy.py, follow_tail.py
        ├── hamilton.py, hamilton_zigzag.py, hamilton_short_cycles.py
        ├── lookahead.py, lookahead_n.py, minimax.py, max_safety.py
        ├── dqn.py, ppo.py, neat_strategy.py
        └── base.py
```

7.4 Útkeresés és „étel felé” stratégiák

7.4.1 A* (`astar`, `AStarStrategy`)

Cél: a fejtől az ételig **legrövidebb út** (lépésszám) a fal és a kígyó teste (farok kivételével akadályként) mellett.

A* prioritási sorral dolgozik; a prioritás $f = g + h$, ahol g a starttól mért lépésszám, h a **Manhattan** távolság az ételtől (engedélyezett irányok: négy szomszéd).

Fallback, ha nincs út vagy az első lépés nem adható meg biztonságosan: (1) **farok felé** útvonal (`path_to_tail` – szintén A*, a farok a cél, a test többi része akadály); (2) ha ez sem jó: **safest_local_step** – minden nem 180°-os szomszédra **flood fill**: hány üres cella érhető el onnan; a **legnagyobb szabadságfok** nyer.

Közös segédfüggvények: `astar_path`, `flood_fill_count`, `direction_from_to` – több más stratégia is importálja őket.

7.4.2 BFS (bfs, BFSStrategy)

Logika megegyezik az A* céljával: szélességi keresés az ételig, **egyenletes lépésköltség** mellett → szintén legrövidebb út (itt nincs heurisztika). **Ugyanaz a fallback** mint az A*-nál: BFS út hiánya → farok út → `safest_local_step`.

Szerepe a projektben: egyszerű, jól értelmezhető **baseline** az A* mellett; ugyanazon a gráfon mérhető a futási költség különbség.

7.4.3 Farok-követés (follow_tail, FollowTailStrategy)

Elsődleges: ha van étel, **A*** út az ételig; ha van érvényes első lépés, azt választja. **Ha nincs** étel út vagy nem léphető: **útvonal a farokhoz** (`path_to_tail`), első lépés, **180°-ot kerülve** ahol lehet. **Végső fallback:** `safest_local_step`.

Intuíció: „Körbejárás” – amikor az étel felé menni kockázatos, a kígyó a farok felé haladva tart teret magának.

7.5 Hamilton-jellegű és ciklusos bejárások

7.5.1 Hamilton spirál (hamilton, HamiltonianStrategy)

Előre generált **spirál ciklus** a teljes rácson (`_build_spiral_cycle`): külső peremtől befelé haladó sávok. A ciklus **cache-elve** (`rows`, `cols`) kulccsal. **Alaplépés:** a fej pozíciójának **következő cellája** a cikluson (`next_on_cycle`).

Étel „levágás”: ha van étel és **A*** útvonalak az első lépésre (test farok nélkül akadály), és az biztonságos, **az első A* lépést** választja a ciklus helyett – így rövidíthet a spirálon.

Megjegyzés: ha a spirál hossza nem egyezik `rows*cols`-szal (pl. páratlan cellaszám), a kód fallback ágai (`safest_local_step`) biztosítanak döntést.

7.5.2 Hamilton zigzag (hamilton_zigzag, HamiltonianZigzagStrategy)

Más ciklus-építés: soronként **bal→jobb**, majd **jobb→bal** váltakozva (`_build_zigzag_cycle`) – tipikusan minden cella pont egyszer, zárt bejárás jelleggel ugyanúgy indexelve.

Étel: mint a spirálnál – előnyben **A* első lépés**, ha biztonságos; különben a zigzag ciklus következő cellája.

7.5.3 Hamilton rövid ciklusok (hamilton_short_cycles, HamiltonShortCyclesStrategy)

A pályát **2x2 blokkokra** bontja. Minden blokkban egy **4 cellás kis kör** van definiálva (`block_cycle_cells`). A kígyó a **saját blokkjának** ciklusán lép tovább (`next_in_block_cycle`).

Étel: ha van biztonságos **A*** út első lépése az étel felé (és nem 180°), azt választja; különben a blokk ciklusa; végül `safest_local_step`.

7.6 Biztonság- és előretekintés-alapú heurisztikák

7.6.1 Greedy (greedy, GreedyStrategy)

Egy sor: mindig a **safest_local_step** – azaz a **maximális flood-fill szabadságfokú** szomszéd (180° tiltva). **Nem** optimalizál közvetlenül az étel felé; ezért benchmarkban gyakran **éhezés** dominál, ha a kígyó túl óvatosan „bezárja” magát üres területen belül.

7.6.2 Maximal safety (**max_safety**, **MaxSafetyStrategy**)

Minden nem 180° -os lépésre **szimulál** (**simulate_step**), majd ellenőrzi: van-e **út a fej és a farok között** a következő állapotban (**has_path_head_to_tail** / **path_to_tail**). Csak az ilyen lépések maradnak; közülük a **legnagyobb flood fill** számú irány nyer. Ha egyik sem felel meg: **safest_local_step**.

Intuíció: a kígyó teste „csőként” marad összefüggő – csökkenti a bezáródás esélyét.

7.6.3 Előretekintés 1 lépés (**lookahead**, **LookAheadStrategy**)

Minden megengedett irányra: (1) szimulált fej pozíció; (2) **flood_fill_count** onnan; (3) opcionálisan **Manhattan távolság az ételtől**. **Első szabály:** nagyobb szabadságfok; **döntetlenben** kisebb távolság az ételhez. Nincs explicit A^* – erős baseline a dokumentált mérésekben.

7.6.4 Előretekintés N lépés (**lookahead_3**, **lookahead_5**, **LookAheadNStrategy**)

N (implementációban 1–6 közé clampelve; a regiszter **3** és **5** értéket használ): minden **első irányra** **simulate_step**, majd **további N–1 lépés** szimulációja; minden belső lépésnél a **legjobb evaluate** állapot szerinti greedy választás (**simulate_greedy_n**). Az **evaluate** függvény: **flood_fill_count** a fejnél mínusz $0.5 \times$ étel Manhattan (ha van étel).

A **végző döntés** azt az **első irányt** választja, amelyik a legjobb végző (vagy köztes) értékelést adja. Halál esetén az adott ág értékelése gyengül / None kezelés a kódban.

7.6.5 Minimax (**minimax**, **MinimaxStrategy(depth=2)**)

Horizont: 1 vagy 2 lépés (a regiszter **2**). Minden **első játékos-lépésre** **simulate_step**; ha mélység 2, minden **második** lépésre is (egyszerűsített „ellenség”: minden lehetséges második irány **minimális** értékelése – a kód a **minimálból** épít maximint). Az értékelés megegyezik a lookahead_n **evaluate** jellegével (szabadság + étel távolság). 180° mindkét szinten tiltva.

Korlát: a „második játékos” nem egy külön ügynök, hanem az összes szomszédos folytatás **legrosszabbja** – konzervatív becslés.

7.7 Tanult stratégiák (inferencia a játékállapoton)

A **state_to_observation** (12 float, lásd §8.1) **azonos** a tanítás és a webszolgáltatás között, így a böngészőben küldött pályaméret mellett is konzisztens a bemenet.

7.7.1 DQN (**dqn**, **DQNStrategy**)

Betöltés: **models/dqn_snake.pt** + **dqn_snake_config.json** (vagy **DQN_MODEL_DIR**). **Háló:** MLP a config szerint (alapértelmezés: $12 \rightarrow 128 \rightarrow 128 \rightarrow 4$ **Q** érték). **Inferencia:** **argmax** Q a négy akcióra; az **ellentétes irány** indexe $-\infty$ maszkolással kiesik. **Hiányzó torch vagy fájl:** **GreedyStrategy** fallback (nem „üres” modell – konzervatív helyi lépés).

Részletek és tanítás: [docs/ai_docs/kornyezet_es_dqn_megvalositas.md](#),
[docs/ai_docs/dqn_tanitas_eredmenyek_es_tovabblepesek.md](#).

7.7.2 PPO (ppo, PPOStrategy)

Betöltés: `stable_baselines3.PPO.load` – először `ppo_snake_best.zip`, majd `ppo_snake.zip` a `models/` mappában. `predict(obs, deterministic=True)` → diszkrét akció 0–3 → **ACTION_NAMES** irány.
Ellentétes irány maszkolás a DQN-hez hasonlóan. **Hiányzó SB3 vagy fájl:** Greedy fallback.

Tanítás: [docs/ai_docs/ppo_tanitas_eredmenyek_es_tovabblepesek.md](#).

7.7.3 Neuroevolution / NEAT (neuroevolution, NEATStrategy)

Betöltés: `models/neat_snake_best.pkl` – pickle: **genome** + **neat config fájl** útvonal.
`neat.nn.FeedForwardNetwork.create` → `activate(obs)` → 4 kimenet (logit szerű); **argmax**, ellentétes akció –∞ maszk. **Hiányzó neat-python vagy fájl:** Greedy fallback.

Tanítás: [docs/ai_docs/neat_tanitas_eredmenyek_es_tovabblepesek.md](#).

Összhang a laikus dokumentummal: a [docs/ai_docs/snake_mi_osszefoglalo_es_tanulasok.md](#) szövege helyenként „placeholder”-ként említi a tanult stratégiákat; a **megvalósítás** szerint **van valódi inferencia**, ha a fenti modellek a `models/` könyvtárban elérhetők – ellenkező esetben a **Greedy** fallback fut, ami benchmarkban a tanult név alatt is **heurisztikus viselkedést** ad.

7.8 REST és WebSocket viselkedés

- **GET /health**, **GET /strategies** – stratégia lista / állapot.
- **POST /next** – egy lépés JSON állapotból.
- **WebSocket /ws** – streamelt állapot → válasz: **action** (irány).
- **GET /benchmark/summaries**, **GET /benchmark/plots/{filename}** – a §5.9 szerinti statisztika nézet táplálása; a szervernek a **benchmarks/results/** és **plots/** elérhetőnek kell lennie a fájlrendszerben.

7.9 Szemléltető kódrészlet: A* mag

```
# astar.py – f és g szétválasztva a heapben
while open_set:
    _, g, current = heapq.heappop(open_set)
    if current == goal:
        # újraépítés came_from-ból
        return path
    for n in neighbors(current):
        tent_g = g + 1
        if n not in g_score or tent_g < g_score[n]:
            came_from[n] = current
            g_score[n] = tent_g
            h = manhattan(n, goal)
            heapq.heappush(open_set, (tent_g + h, tent_g, n))
```

7.10 Stratégiák táblázatban (azonosító → fájl)

Azonosító	Osztály / modul	Rövid logika
astar	AStarStrategy	A* ételig → farok / legbiztonságosabb
bfs	BFSStrategy	BFS ételig → ugyanaz a fallback
follow_tail	FollowTailStrategy	A* étel → farok → safest
greedy	GreedyStrategy	Max flood fill
hamilton	HamiltonianStrategy	Spirál ciklus + A* levágás
hamilton_zigzag	HamiltonianZigzagStrategy	Zigzag ciklus + A* levágás
hamilton_short_cycles	HamiltonShortCyclesStrategy	2×2 blokkciklus + A*
lookahead	LookAheadStrategy	1 lépés: szabadság + étel
lookahead_3 / lookahead_5	LookAheadNStrategy	N lépés greedy szimuláció
minimax	MinimaxStrategy(depth=2)	2 lépés maximin
max_safety	MaxSafetyStrategy	Fej-farok út + max flood
dqn	DQNStrategy	Q-háló vagy Greedy
ppo	PPOStrategy	SB3 PPO vagy Greedy
neuroevolution	NEATStrategy	NEAT háló vagy Greedy

8. Tanuló környezet és algoritmusok

Ez a fejezet a **tanításhoz és a tanult stratégiák bemenetéhez** kötődő **SnakeEnv** környezetet és a **DQN / PPO / NEAT** algoritmusok **tanítási logikáját** foglalja össze – összhangban a [docs/ai_docs/kornyezet_es_dqn_megvalositas.md](#), [snake_mi_osszefoglalo_es_tanulasok.md](#), [rl_es_neuroevolution_tanitas_terv.md](#) fájlokkal. A §7 a **döntés pillanatában** futó stratégiákat írja le; ez a fejezet azt, **hogyan készül** a tanult policy / Q-háló / NEAT genom.

8.1 SnakeEnv – megfigyelés és jutalom

Fájl: `ai_service/src/env/snake_env.py`. **Interfész:** `reset(seed=None) -> (observation, info)`, `step(action: int) -> (observation, reward, done, info)`. **Akciók:** 0=Up, 1=Right, 2=Down, 3=Left (`ACTION_NAMES`, `DIR_TO_ACTION`).

Megfigyelés: `state_to_observation` → **12 float** (`OBSERVATION_DIM = 12`), ugyanaz, mint a §7.7 inferenciánál:

Index	Tartalom
0–3	Veszély (Up, Right, Down, Left): 1.0, ha fal vagy test 1–2 lépésben

Index	Tartalom
4–7	Fal távolság négy irányban, normalizálva <code>max(rows, cols)</code> szerint
8–9	Étel relatív: <code>(food.x - head.x)/cols</code> , <code>(food.y - head.y)/rows</code> ; nincs étel → 0,0
10	Ételtől Manhattan / <code>(rows+cols)</code> ; nincs étel → 1.0
11	Kígyóhossz / <code>(rows*cols)</code>

Jutalom (konstruktor paraméterezhető): alapértelmezés szerint étel `reward_food`, halál `reward_death`, túlélés `reward_survival` minden lépésben, Manhattan `reward_step_toward / reward_step_away` ha nem evett. Ha be van állítva `max_steps_per_episode`, elérésekor `done=True` és `reward_starvation` hozzáadódik (epizód időkorlát / éhezés jelleg).

Étel: a `simulate_step` étkezés után `food=None`-t ad; a környezet **új ételt** spawnol üres cellán a saját **Random** generatorával (seed → reprodukálhatóság).

8.2 DQN (Deep Q-Network) – tanítás és inferencia összefoglalva

Ötlet: neurális háló becsüli $Q(s,a)$ értékeket mind a négy irányra; a politika tanítás közben **ϵ -greedy** (felfedezés → kihasználás), cél a **TD-hiba** minimalizálása egy **replay buffer** mintáiból; **target network** periodikus másolása a stabilitásért.

Implementáció: `training/train_dqn.py` – PyTorch MLP, buffer, epsilon csökkenés, mentés `dqn_snake.pt` + `dqn_snake_config.json`. **Inferencia:** §7.7.1.

Olvasható összefoglaló: `snake_mi_osszefoglalo_es_tanulasok.md` §6; technikai táblázatok: `kornyezet_es_dqn_megvalositas.md` §2.

8.3 PPO (Proximal Policy Optimization)

Ötlet: közvetlenül **stochastikus politika** $\pi(a|s)$ tanítása (diszkrét akcióknál négy valószínűség / logit), **value függvény** $V(s)$ a **GAE** / advantage becsléséhez; **klipelt** policy loss, hogy a frissítés ne legyen túl nagy egy lépésben („proximal”).

Implementáció: `training/train_ppo.py` + **Stable-Baselines3** **PPO** osztály, Gym-kompatibilis becsomagolt env; mentés `.zip`. **Inferencia:** §7.7.2.

Laikus leírás: `snake_mi_osszefoglalo_es_tanulasok.md` §7; eredmények és hangolás: `ppo_tanitas_eredmenyek_es_tovabblepesek.md`.

8.4 NEAT (NeuroEvolution of Augmenting Topologies)

Ötlet: populáció **genomjai** (súlyok + opcionálisan topológia) **fitness** szerint szelektálódnak; nincs backprop a klasszikus RL értelemben – a **szabályrendszer** (NEAT) mutáció, reprodukció, fajok (`neat-python`).

Implementáció: `training/train_neat.py` – fitness = játékbeli jutalom / pont; legjobb genome mentése `neat_snake_best.pkl` + config útvonal. **Inferencia:** §7.7.3.

Laikus leírás: `snake_mi_osszefoglalo_es_tanulasok.md` §8; további lépések: `neat_tanitas_eredmenyek_es_tovabblepesek.md`.

8.5 Tanítás, inferencia és benchmark egységes nézőpontból

- **Ugyanaz a megfigyelés-vektor** köti össze a tanítást, a DQN/NEAT inferenciát és az SB3 PPO **Box** observation space-t (a becsomagoló kód a projektben a 12 dimenzióhoz igazít).
- **Eltérő pályaméret** a tanításhoz képest: a vektor normalizált komponensei miatt **futhat**, de a **legjobb teljesítmény** a tanítási rácsmérethez (tipikusan **20×20**, lásd **kornyezet_es_dqn_megvalositas.md**) kötődik.
- **Benchmark:** **run_strategy_benchmark.py** ugyanazokat a **STRATEGIES** kulcsokat futtatja, mint a játék – a JSON eredmények a **\$5.9** statisztika oldalon jelennek meg.

9. Benchmark: protokoll és eszközök

Ez a fejezet a **mérési protokollt** és az **eszközláncot** rögzíti – összhangban a **benchmarks/run_strategy_benchmark.py** forrásával, a **docs/ai_docs/benchmark_eredmenyek_reszletes_kiertekeles.md**, **benchmark_kiertekeles_es_adatvizualizacio_utmutato.md** és **szakdolgozat_statisztika_es_elemzes_struktura.md** dokumentumokkal, valamint a **\$5.9** webes Statisztika oldal által feltételezett adatstruktúrával.

9.1 Cél és szerep

A benchmark célja **objektív, reprodukálható összehasonlítás** több stratégia között: ugyanaz a pálya, ugyanaz a kezdőállapot-függő **seed-sorozat**, ugyanaz a lépés- és éhezési limit. Nem „ember elleni verseny”, hanem **döntési logikák** (heurisztika vs. betanított modell) viselkedése **statisztikai mintán** – ahogy a frontend **Statistics.tsx** bevezető szövege is kiemeli.

9.2 Futtató: **run_strategy_benchmark.py**

Belépési pont: projekt **gyökeréből** érdemes futtatni (a script a **ROOT**-ot a **benchmarks/** szülőjére állítja, és az **ai_service** csomagot teszi a **sys.path**-ra).

Egy játék (run_one_game**):**

1. **STRATEGIES[**strategy_key**]** példányosítása (osztály vagy lambda → mindkettő hívható konstruktorként).
2. **Kezdőállapot:** **create_initial_state(rows, cols, seed)** – középen **3 cellás** kígyó, irány **Jobbra**, étel véletlen üres cellán ugyanazzal a **seed**-del inicializált **Random**-gal.
3. **Ciklus:** minden lépésben **strategy.next_move(state)**; ha a stratégia a **180°-os** fordulatot adná, a futtató **megtartja az aktuális irányt** (ütközés elkerülése a specifikus edge case-re).
4. **step:** fal / önmaga → **game_over** + **wall** vagy **self**; étel evés → növekvő score, új étel.
5. **Éhezés (starvation):** ha **steps_since_food >= 400** (a forrásban fix konstans: „20×20 pályán 400 lépés érdemi éhezési limit”), a kör **starvation** okkal leáll – így a végtelen „toporgás” nem nyújtja a benchmarkot.
6. **Lépéslimit:** ha eléri a **max_steps** értéket ütközés nélkül, a halál oka a JSON-ban tipikusan **max_steps** (a kód: **death or "max_steps"**).

CLI paraméterek (alapértelmezés): **--runs 200, --rows 20, --cols 20, --max-steps 5000, --seed-base 42, --strategies** vesszős lista; ha **--strategies** nincs megadva, egy **beépített alapkészlet** fut

(greedy, max_safety, hamilton, astar, dqn, ppo, neuroevolution). **Kimeneti könyvtár:** `--out-dir` (alpból `benchmarks/results/`).

Parancs példa (a dokumentált kampány szerint, lásd `benchmark_eredmenyek_reszletes_kiertekeles.md`):

```
python benchmarks/run_strategy_benchmark.py --runs 500 --strategies
greedy,hamilton,max_safety,astar,bfs,follow_tail,lookahead,lookahead_3,lookahead_5,minimax
python benchmarks/run_strategy_benchmark.py --runs 200 --strategies
dqn,ppo,neuroevolution
```

9.3 Kimenet: JSON és összefoglaló

Minden stratégiára külön fájl: `benchmarks/results/strategy_benchmark_<azonosító>.json`, szerkezet: **summary** + **runs** (soronkénti `score`, `steps`, `death`, `seed`).

A **summary** mező tartalmazza: `strategy`, `runs`, `rows`, `cols`, `max_steps`, `seed_base`, **score_mean**, **score_median**, **steps_mean**, **steps_median**, **death_counts** (kulcsonkénti darabszám), **reached_first_food** (százalék: futások hány százalékában **score** ≥ 1 , azaz legalább egy étel).

A futás végén felülírt fájl: `strategy_benchmark_summary.json` – stratégiánként csak a **summary** objektumok (a webes `/benchmark/summaries` ebből vagy a külön JSON-ekből épít).

Módszertani megjegyzés (`benchmark_eredmenyek_reszletes_kiertekeles.md`, `benchmark_kiertekeles_es_adatvizualizacio_utmutato.md`): a **különböző runs** (pl. 500 vs 200) **nem érvényteleníti** az összehasonlítást, de a dolgozatban **explicit jelezni** kell; publikációs szinten ajánlott egy **közös kontroll** (pl. minden stratégia **200** futás ugyanazzal a protokollal).

9.4 Vizualizáció: `generate_benchmark_plots.py`

Futtatás: `python benchmarks/generate_benchmark_plots.py` (`matplotlib` szükséges). Beolvassa az összes `strategy_benchmark_*.json`-t (a `strategy_benchmark_summary.json` **kizárva**), és a `benchmarks/results/plots/` mappába menti többek között:

Fájl	Értelmezés
<code>score_mean_bar.png</code>	Gyors rangsor átlagpont szerint
<code>score_distribution_boxplot.png</code>	Stabilitás, szórás, outlierok
<code>death_distribution_stacked.png</code>	Halálok megoszlása (<code>wall</code> / <code>self</code> / <code>starvation</code> / <code>max_steps</code>)
<code>score_vs_steps_scatter.png</code>	Pont vs. lépésszám trade-off

Ezek a fájlok a **§5.9** szerinti Statisztika oldalon is megjeleníthetők (`/benchmark/plots/...`).

9.5 Régi benchmark és további olvasmány

- **run_benchmark.py**: korai, főleg **A*** vs **Hamilton** jellegű összehasonlításra; a szakdolgozat **egységes** összképéhez a **run_strategy_benchmark.py** az irányadó. Részletek: **benchmarks/README.md**.
- **Elemzési vázlat** (további fejezet- és metrikaötletek): **docs/ai_docs/szakdolgozat_statisztika_es_elemzes_struktura.md** (mean vs median, fair comparison, modellverzió rögzítése).

10. Mérési eredmények és értelmezés

Az alábbi táblázatok egy **konkrét, dokumentált kampány** számai (**docs/ai_docs/benchmark_eredmenyek_reszletes_kiertekeles.md**); a repó **benchmarks/results/*.json** fájljai a **frissíthető forrás**. A **webes Statisztika** oldal (**Statistics.tsx**) ugyanezekből a típusú mezőkből dolgozik, és **szöveges útmutatót** ad a metrikák értelmezéséhez – az alábbi alfejezetek ezt és a **strategyBenchmarkDetail.ts** stratégiánkénti magyarázatait foglalják össze szakdolgozati szinten.

10.1 Mit jelentenek a metrikák? (összhang a webes Statisztikával)

- **runs**: mintanagyság – nagyobb érték **stabilabb átlag**, hosszabb futási idő.
- **Átlag vs. medián pont**: az **átlag** érzékeny a szélsőértékekre; a **medián** a „tipikus” játékot jól jelzi – rangsoroláskor mindkettőt érdemes nézni (**kerülendő az „átlagfetisizmus”** – **benchmark_kiertekeles_es_adatvizualizacio_utmutato.md**).
- **Átlag lépés**: hosszabb játék gyakran jobb túlélést vagy hosszabb „keringést” is jelenthet; **önmagában** nem egyenlő a jobb stratégiával.
- **Halál okok (wall, self, starvation, max_steps)**: a futtató szerinti besorolás. Az **éhség (starvation)** a weben is hangsúlyozva: **nem** valós játékbeli ütközés, hanem azt jelenti, hogy **sok lépésen át nem nőtt a pontszám** (400 lépés limit), majd a benchmark leállította a kört.
- **Hiányzó halál-kulcs a JSON-ban**: a frontend **0%-ot** mutat (nem „hiányzó adat”).
- **Első étel elérése (reached_first_food)**: ha alacsony, a stratégia sok futásban **nem jut el stabilan az első ételig** – robusztusság mutatója.

10.2 Heurisztikák vs. tanult stratégiák (a weboldal narratívája)

A Statisztika oldal **„Heurisztikák vs. tanult stratégiák”** blokkja szerint:

- A **heurisztikák** explicit szabályokat követnek (útkeresés, előretekintés, farok-követés stb.), ezért ebben a protokollban gyakran **magasabb átlagpontot** adnak, ha a szabály jól illeszkedik a pályaméterhez.
- A **tanuló** (RL / evolúciós) rendszerek ugyanazon felületen sokszor **gyengébben indulnak**: jutalom- és állapot-reprezentáció finomhangolása nélkül könnyen kialakul **„biztonságos toporgás”** vagy instabil viselkedés – ez sok **starvation** jellegű leállásban tükröződik.
- **Kulcs mondat a UI-ban**: *„Rosszabb benchmark nem jelenti automatikusan, hogy a módszer rossz”* – csak azt, hogy **ebben a környezetben / beállításban** még nem versenyképes. A heurisztikák **baseline**-ként szolgálnak: ha egy tanult ügynök nem közelíti meg őket, érdemes először a **jutalmat**, az **epizód hosszát** és a **megfigyelést** finomítani.

10.3 Halálprofilok és szakmai értelmezés

A **benchmark_eredmenyek_reszletes_kiertekeles.md** szerint:

- **Greedy / MaxSafety / DQN** (ebben a mérésben): gyakorlatilag **csak starvation** – nem feltétlen „jó túlélés”; inkább azt jelzi, hogy a lépések **nem konvertálódnak** hatékonyan pontszerzésre.
- **PPO / NEAT**: domináns **starvation**, de már jelen van **self / wall** is → **aktívabb**, kockázatosabb mozgás.
- **A*, BFS, FollowTail, Hamilton, Lookahead**: főként **self** és **wall**; **starvation** elhanyagolható → a stratégiák **aktívan keresik az ételt**, és ütközésben halnak meg inkább, mint éhen.

Összefüggés: magasabb **self** arány gyakran **agresszívebb ételkereséssel** társulhat (több kockázat ↔ több pont lehetősége) – ezért a halálprofilt **a pontszámmal együtt** kell olvasni.

10.4 Hatékonyság és robusztusság (összehasonlító keret)

A **benchmark_kiertekeles_es_adatvizualizacio_utmutato.md** javasolt kerete:

- **Hatékonyság:** score (mean / median szerinti rangsor), első étel %.
- **Robusztusság:** magas **reached_first_food** + magas medián pont + viszonylagosan kiegyensúlyozott halálmegoszlás.
- **Trade-off:** a **score_vs_steps_scatter** diagram: ki szerzi „**drágán**” a pontot (sok lépés, alacsony score).

A fenti szempontokhoz illeszkedő **generált grafikonok** (oszlopdiagram, boxplot, halálmegoszlás, szórásdiagram) a **§10.7** alfejezetben láthatók beágyazott PNG-ként.

10.5 Heurisztikák (500 futás / stratégia, dokumentált kampány)

Stratégia	Átlag pont	Medián	Átlag lépés	Első étel (%)
lookahead	92.75	92	2460.71	100.0
astar	77.20	77	1297.47	100.0
follow_tail	77.20	77	1297.47	100.0
bfs	74.40	76	1245.71	100.0
hamilton	57.01	57	851.08	100.0
lookahead_3	10.93	10	180.68	100.0
minimax	4.41	3	446.56	80.6
lookahead_5	0.30	0	403.73	15.2
greedy	0.20	0	404.62	15.0
max_safety	0.20	0	404.62	15.0

Rövid elemzés: a **1 lépéses lookahead** a legerősebb heurisztika (kimagasló átlag és medián, 100% első étel).

A*, follow_tail, BFS stabil felső-közép mezőny. **Hamilton** erős, de elmarad a legjobb útvonalas / lookahead megoldásoktól. **Greedy, max_safety, lookahead_5** ebben a protokollban **éhezés-domináns**, nagyon alacsony score – összhangban a **§7.6** leírással (túl óvatos vagy túl mély szimuláció más hibamintázattal).

10.6 Tanuló stratégiák (200 futás / stratégia)

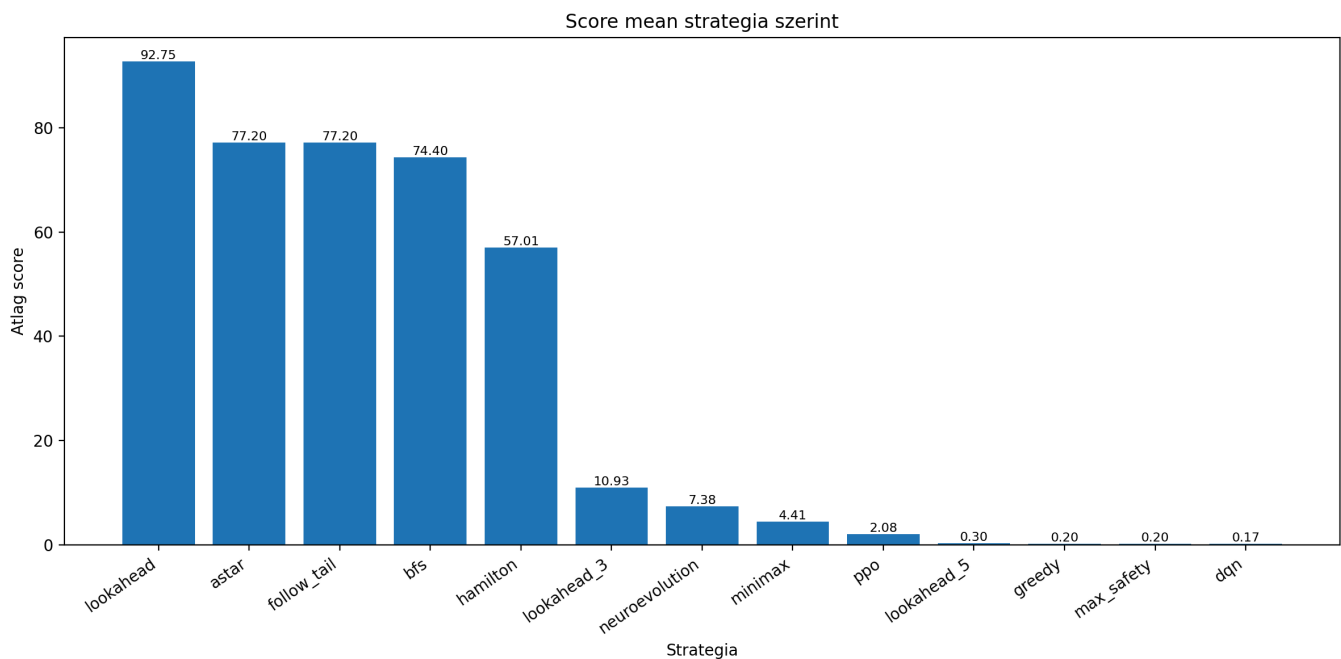
Stratégia	Átlag pont	Medián	Átlag lépés	Első étel (%)
neuroevolution (NEAT)	7.38	5	1027.16	90.5
ppo	2.08	2	384.67	66.0
dqn	0.17	0	403.36	12.5

Rövid elemzés: NEAT > PPO > DQN ezen a kampányon; a NEAT közelebb kerül a heurisztikákhoz, de még **nem éri el** a top heurisztikákat. A **DQN** ebben a setupban **erős starvation** profillal gyenge – összhangban a tanítási dokumentációval (jutalom, timesteps, modellverzió). **Korlát:** a tanult modellek **konkrét fájlverziói** rögzítendőek a reprodukálhatósághoz ([benchmark_kiertekeles_es_adatvizualizacio_utmutato.md](#) §1.2).

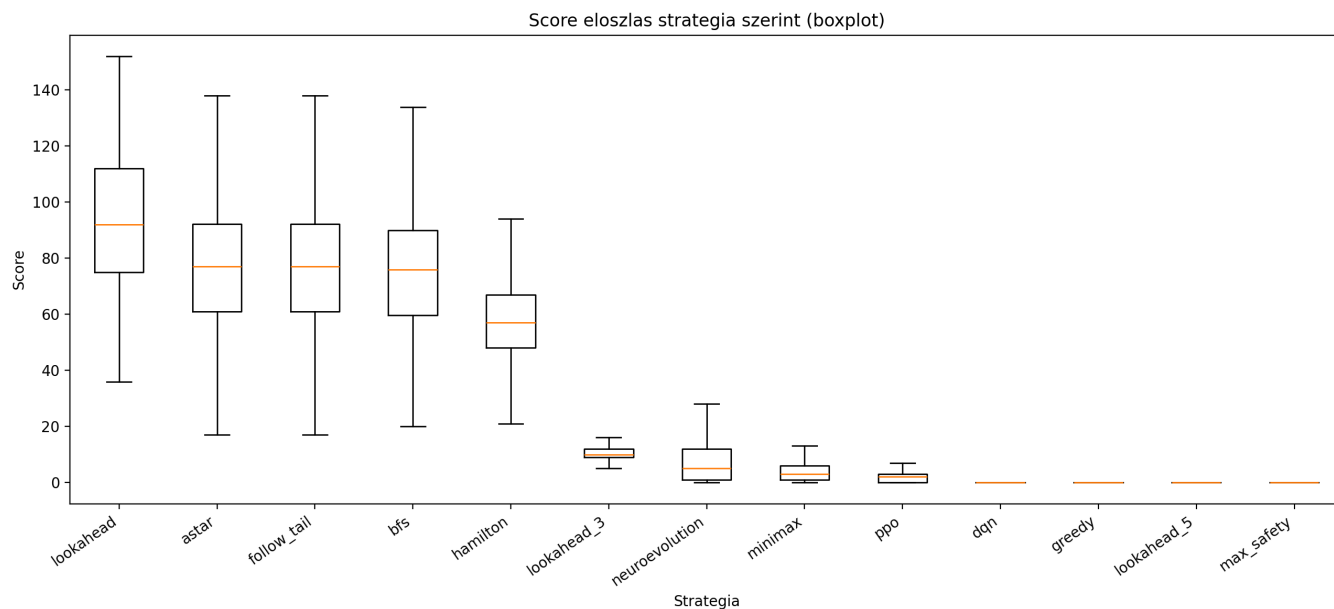
10.7 Vizualizáció: generált grafikonok (PNG)

Az alábbi ábrák a [benchmarks/generate_benchmark_plots.py](#) szkripttel készülnek a [benchmarks/results/strategy_benchmark_*.json](#) fájlokból; a fájlok a repó [benchmarks/results/plots/](#) mappájában vannak. A Markdown a [docs/](#) mappához képest **relatív útvonalon** hivatkozik rájuk (előnézet / PDF export esetén a renderelőnek el kell érnie ezeket a fájlokat). Ugyanezek a képek a webes **Statistika** nézeten is megjeleníthetők ([§5.9](#)).

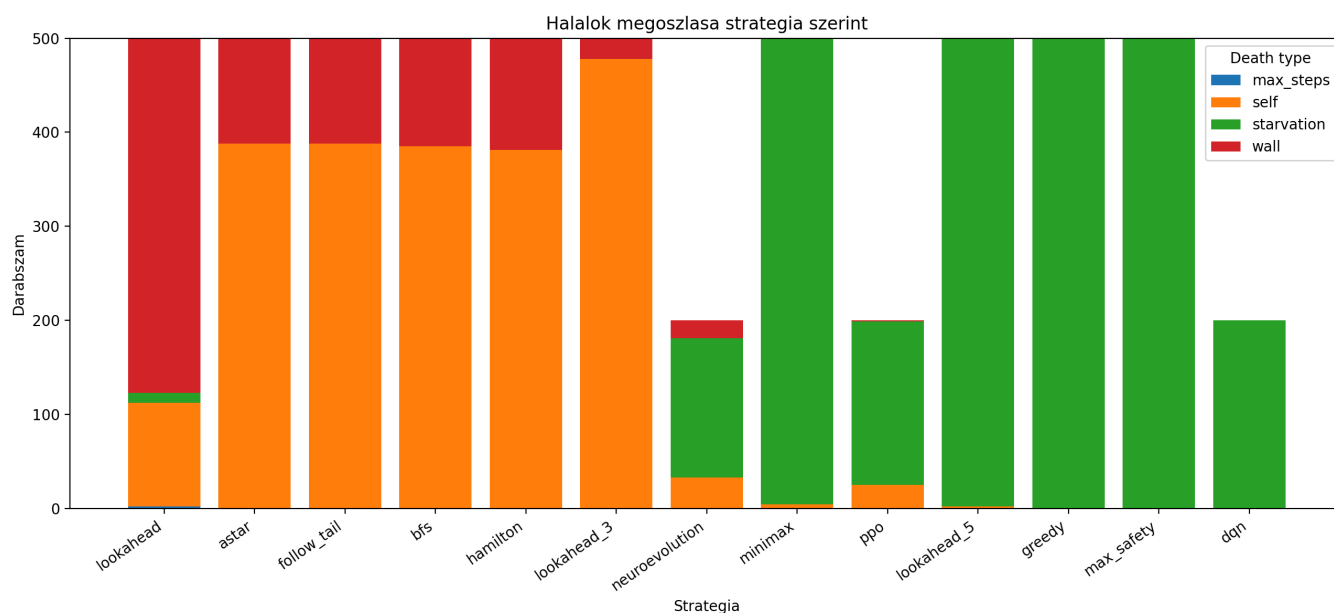
Átlagpont stratégiánként – gyors rangsorolás:



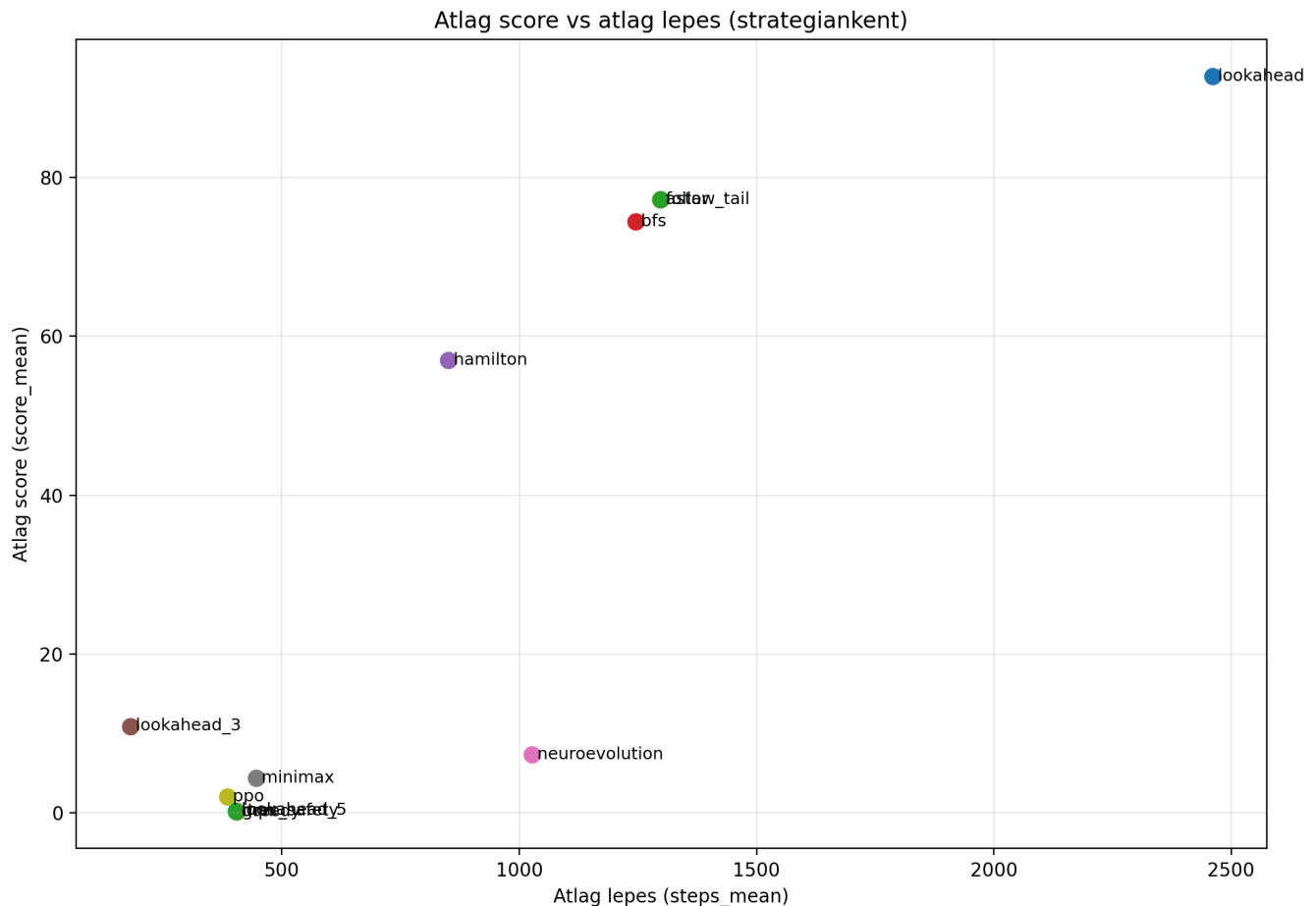
Ponteloszlás – medián, kvartilisok és kiugrók (boxplot):



Halálok megoszlása – wall / self / starvation / max_steps arányok (stacked):



Pont vs. lépésszám – hatékonyság / túlélés trade-off (szórásdiagram):



10.8 Záró következtetések (kampány + UI + szakmai útmutató)

1. A dokumentált benchmark szerint a **legerősebb stratégiák heurisztikusak** (különösen **lookahead**, **astar**, **follow_tail**, **bfs**, **hamilton**).
2. A tanult módszerek közül a **NEAT** adja a legjobb kompromisszumot, de **nem múlja felül** a top heurisztikákat.
3. A **PPO** és főleg a **DQN** ebben a konfigurációban **alulteljesít**; a halálprofil és az első étel % **összhangban** van a webes magyarázattal (éhség / toporgás).
4. Tudományos érték: **azonos környezetben a paradigma** (kézi heurisztika vs RL vs neuroevolution) **erősen befolyásolja** a mérhető teljesítményt – ez nem „általános algoritmus-rangsor”, hanem **fix protokoll melletti** összehasonlítás.
5. **Javasolt kontrollfutás** (idézet a részletes kiértékelésből): egy parancsban **minden** stratégia **azonos runs** értékkel (pl. 200), hogy a mintanagyság ne torzítson.

11. Webes alkalmazás: funkcionalitások (képernyők, auth, játék)

Ez a fejezet a **felhasználói élmény** szempontjából írja le, mit tud a webes alkalmazás: mely **képernyők** váltanak egymásra, milyen **adatok** és **háttér szolgáltatások** (Node backend, AI szolgáltatás) kapcsolódnak hozzájuk. A megvalósítás fájljai: főleg **frontend/src/App.tsx**, **frontend/src/ui/***, **frontend/src/view/***, **frontend/src/hooks/***, **frontend/src/api.ts**, **frontend/src/io/***.

11.1 Áttekintés: egyoldalas navigáció és **screen** állapot

Az alkalmazás **egyoldalal (SPA)** felület: a gyökérkomponens egy **screen** React állapot szerint vált a nézetek között. A lehetséges értékek típusa:

```
type Screen = 'menu' | 'settings' | 'results' | 'statistics' | 'game' | 'login'
| 'register' | 'profile'
```

- **Menü és környezet:** **menu**, **settings**, **results**, **statistics** – mindegyiknél megjelenik a közös fejléc (**Header**).
- **Auth:** **login**, **register** – szintén fejléccel; siker után tipikusan **menu**.
- **Profil:** **profile** – csak akkor érdemes megnyitni, ha a felhasználó be van jelentkezve (a menü és a fejléc is ezt feltételezi).
- **Játék:** **game** – ekkor a fejléc alatt **HUD**, **Canvas**, **vezérlő gombok** és (INIT fázisban) **Start** réteg látható; nincs külön „játék route”, csak ez az állapot.

Párhuzamosan a játékhoz tartozik a **config** (**GameConfig**, **loadConfig** induláskor) és a **gameMode**: **'player' | 'ai'**. A főmenü rádiója és a beállítások **MI** füle határozza meg, melyik módot indítja el a „Játék indítása” / mentés utáni indítás.

11.2 Navigáció és fejléc (**Header**)

A **Header** minden fenti nézetben látható (kivéve, hogy a játék képernyőn is megjelenik a fejléc a HUD felett). Funkciói:

- **Cím:** „Snake – MI” (alkalmazás azonosítás).
- **Auth:** kijelentkezve **Bejelentkezés** és **Regisztráció** gombok → **login** / **register** képernyő. Bejelentkezve **felhasználónév**, **Profil**, **Kijelentkezés** (**AuthContext.logout()** – törli a **snake_token** és **snake_user** kulcsokat a localStorage-ból; a játék-konfiguráció és a téma **nem** törlődik).
- **Téma:** váltógomb a **ThemeContext.toggleTheme**-re kötve (részletek: 11.5).

A fejléc callbackjei az **App.tsx**-ben egy helyen vannak összerakva (**onLoginClick**, **onRegisterClick**, **onProfileClick**), így minden képernyő egységesen ugyanazt az auth navigációt kapja.

11.3 Főmenü (**MainMenu**)

A **MainMenu** a belépési pont indulás után (**screen === 'menu'**).

- **Játék mód választó (rádió):** **Játékos** vagy **MI**. Ez **nem** írja felül önmagában a **config** mezőit; csak azt határozza meg, hogy a „Játék indítása” gomb melyik móddal hívja az **onStartGame(config, mode)**-ot. A pályaméret, tick, seed és MI stratégia a **Beállításokban** állítható (lásd 11.6).
- **Játék indítása:** **startGameWithConfig** logika: **setConfig** + **saveConfig** → **createGame** a megadott seeddel (**config.seed** hiányában **Date.now()**), **gameMode** beállítása, **screen** → **'game'**.
- **Beállítások:** **screen** → **'settings'**.
- **Eredmények:** ha van **JWT (token)**, az **App** meghívja a **fetchScores()** API-t; a válasz **ScoreEntry** sorait **StoredScore** formára mapeli (köztük **ai_strategy** → **aiStrategy**), **setScores**, majd **results**. Hiba esetén üres lista és mégis megnyílik az eredmények nézet. **Vendég** módban nincs hálózati hívás: **loadScores()** a localStorage-ból.
- **Statisztika:** **screen** → **'statistics'** (benchmark adatok; részletesen 5.9).
- **Profil:** csak **bejelentkezve** jelenik meg a menüben; **screen** → **'profile'**.

11.4 Bejelentkezés, regisztráció és auth perzisztencia

Bejelentkezés (`LoginForm`): felhasználónév **vagy** email + jelszó. A kérés a **api.login**-on megy (`VITE_API_URL` vagy `http://localhost:3000`). Siker esetén **setAuth(token, user)** (`AuthContext`): `localStorage.setItem('snake_token', ...)`, `localStorage.setItem('snake_user', JSON.stringify(user))`, majd navigáció a menüre.

Regisztráció (`RegisterForm`): email, felhasználónév, jelszó, megerősítés; validáció és duplikáció a backend válasza szerint. Siker után ugyanúgy token + user tárolás és menü.

Mindkét űrlapon **Vissza** → **menu**. Az **AuthProvider** induláskor visszaolvassa a token/user párost; ha van token, de a user hiányzik, **getMe()** próbál szinkronizálni – hiba esetén **logout()**.

11.5 Témaválasztás (világos / sötét)

A **ThemeContext** a `document.documentElement` **data-theme** attribútumát állítja (**light** / **dark**), és a választást **snake_theme** kulccsal menti a `localStorage`-ba. A színek a **theme.css** szelektorai alapján változnak (`[data-theme="dark"]` / `[data-theme="light"]`).

Fontos megkülönböztetés: a **GameCanvas** 2D kontextusban **közvetlenül** kapja a téma alapján választott színeket (nem CSS változókból olvas), mert a Canvas nem öröklí automatikusan a stíluslap változóit. Így a téma váltás a teljes UI-ra és a játéktér palettájára is hat.

11.6 Beállítások (**Settings**, **PlayerSettings**, **AISettings**)

A beállítások képernyő **két fület** használ (`role="tablist" / tabpanel`):

1. **Játékos** (**PlayerSettings**): **sor- és oszlopszám** (ésszerű tartomány, tipikusan 10–40), **tick időköz** ms-ban (játéksebesség), opcionális **seed** (üres = véletlen indulás mentéskor/indításkor). **Mentés** → **onSave** (`saveConfig` + `setConfig`). **Játék indítása** → játékos módban azonnali indítás ugyanezzel a konfigurációval.
2. **MI** (**AISettings**): ugyanazok a pálya / tick / seed mezők, plusz **stratégia** legördülő: az **AI_STRATEGIES** lista (`strategies.ts`) minden eleméhez név és rövid leírás; a kiválasztott azonosító a `config.ai.strategy` mezőbe kerül, és **localStorage**-ba mentve marad.

Alul **Vissza a főmenübe** zárja a nézetet. Az induláskor betöltött konfiguráció **loadConfig** (`io/config.ts`).

11.7 Játék képernyő – játékos mód

HUD (`HUD.tsx`): élő **pont**, **kígyó hossz**, **lépésszám**, **tick/s** (`1000 / tickMs`), **státusz** (INIT → „Készül”, RUNNING → „Fut”, PAUSED → „Szünet”, GAME_OVER → „Vége”). MI módban kiegészül egy **MI**: sorral (lásd 11.8).

Canvas: a pálya, kígyó és étel rajzolása (**GameCanvas**).

Billentyűzet (globális `keydown` az `App.tsx`-ben, csak `screen === 'game'` esetén):

- **Nyilak** és **WASD**: irány (`setDirection`) – **játékos** módban, ha a fázis **RUNNING** vagy **INIT**.
- **P**: szünet / folytatás (`pauseGame` / `resumeGame`), ha **RUNNING** vagy **PAUSED**.
- **R**: új játék **ugyanabban** a módban (`startNewGame(gameMode)`).
- **Enter**: **INIT** fázisban `startGame`.

Egér: INIT alatt egy **Start** gomb is megjelenik átfedésként (`game-start-overlay`); ez ugyanazt csinálja, mint az Enter.

Gombok a játék alatt: **Főmenü**, futás közben **Pause (P)**, szünetben **Folytatás (P)**, **Új játék (R)**. GAME_OVER után szöveg: „**Játék vége. Pont: ...**”.

Játékciklus: `useGameLoop` csak akkor aktív, ha `screen === 'game'` és `gameMode === 'player'` – fix intervallumos `tick` hívások.

11.8 Játék képernyő – MI mód

Ugyanaz a HUD és Canvas, de a léptetést `useAIGameLoop` végzi, ha `screen === 'game'` és `gameMode === 'ai'`. Minden tick után a kliens **WebSocket**en elküldi a pillanatképet (`VITE_AI_WS_URL` vagy `ws://localhost:8000/ws`), stratégia azonosítóval; a válasz **irány** alapján történik a mozgás. Ha nincs kapcsolat, **helyi fallback** (`placeholderStrategy` a `Strategy.ts`-ben).

A HUD ekkor a következőt jeleníti meg (a `getStrategyName` a felhasználóbarát nevet adja):

```
{aiConnected !== undefined && (
  <span title={aiConnected ? 'ai_service WebSocket' : 'Helyi stratégia'}>
    MI: <strong>{aiConnected ? `backend (${getStrategyName(aiStrategy)})`
  : 'helyi'}</strong>
```

11.9 Játék vége és pontszám mentés

Amikor a játékállapot **GAME_OVER** és a képernyő még **game**:

1. **saveScore** (`io/storage.ts`): pont, tick, hossz, ISO dátum, mód (`player / ai`), MI esetén `aiStrategy`.
2. **setScores**(`loadScores()`) – a lokális lista frissítése.
3. Ha van **JWT**, **submitScoreApi** (POST `/api/scores`) – a szerver oldali eredménytáblába; hiba esetén csendes `.catch(() => {})`.

Fejlesztői megjegyzés: az `App.tsx` egy `useEffect`-ben induláskor meghívja `clearScores()`, ami **törli a vendég localStorage eredménylistáját**. Éles vagy demonstrációs használat előtt érdemes eldönteni, ezt megtartjuk-e; különben az oldal újratöltése után a lokális eredmények nem maradnak meg.

```
useEffect(() => {
  clearScores()
}, [])
```

11.10 Eredmények oldal (`Results`)

Az **Results** a főmenü **Eredmények** gombjáról nyílik; propként kapja a már betöltött `scores` tömböt (az `App` intézi a backend vs. localStorage forrást, lásd 11.3).

- Bejelentkezett felhasználónál a lista felett „**Felhasználó: ...**” megjelenik.

- Minden sor: **pont, lépés, hossz, Játékos** vagy **MI (stratégia megjelenített neve), helyi idő** (`toLocaleString('hu-HU')`).
- Legfeljebb **20** sor (`scores.slice(0, 20)`).
- Üres lista: „**Még nincs mentett eredmény.**”
- **Vissza** a menüre.

11.11 Statisztika oldal (**Statistics**) – összefoglaló

A benchmark eredmények böngészős megjelenítése: **HTTP** az AI szolgáltatás **/benchmark/summaries** és **/benchmark/plots/...** végpontjaira, táblázat szűréssel és rendezéssel, részletes stratégia panel, PNG grafikonok és lightbox. **Részletes funkcionalitás:** §5.9 – Statisztika oldal és adatvizualizáció.

11.12 Profil oldal (**Profile**)

Csak **bejelentkezett** felhasználónak érdemes megnyitni (a menü így is kínálja). Funkciók:

- **Felhasználónév módosítása** – `updateUsername`, majd `loadUser()` a friss adatokért.
- **Jelszó módosítás** – jelenlegi / új / megerősítés, kliens oldali ellenőrzések; `updatePassword`.
- **Saját eredmények** – `fetchScores()` ugyanazzal a megjelenítési logikával, mint az eredményeknél.

Vissza a főmenüre.

11.13 Adatforrások és szolgáltatások (összehasonlító táblázat)

Funkció	Elsődleges tárolás / forrás
Téma	<code>localStorage</code> (<code>snake_theme</code>), <code>ThemeContext.tsx</code> , <code>theme.css</code>
Játék konfiguráció	<code>localStorage</code> , <code>io/config.ts</code>
Eredmények (vendég)	<code>localStorage</code> , <code>io/storage.ts</code> (induláskor <code>clearScores</code> miatt ürülhet)
Auth	JWT <code>snake_token</code> , user JSON <code>snake_user</code> ; REST backend :3000
MI lépés játék közben	WebSocket → <code>ai_service</code> :8000 /ws
Statisztika táblázat / grafikon	HTTP → :8000 /benchmark/*; fájlok <code>benchmarks/results/</code>

12. Korlátok és következtetések

Ez a fejezet **összefoglalja a megállapítások hatókörét** és a **fő korlátokat**: mit szabad és mit nem érdemes levonni a dokumentált mérésekből és a webes Statisztika nézetből. Összhangban áll a 10. fejezet záró alfejezetével (§10.8), a `docs/ai_docs/benchmark_kiertekeles_es_adatvizualizacio_utmutato.md` §3–4 (jó / kerülendő következtetések, interpretációs hibák) részével, valamint a `Statistics.tsx` szövegével („rosszabb benchmark nem jelenti automatikusan, hogy a módszer rossz”).

12.1 Mérési és módszertani korlátok

- **Egy pálya, egy protokoll:** a benchmark **20×20**, fix `seed_base`, `max_steps`, **400 lépéses éhezési cutoff** és adott `runs` mellett értelmezhető (részletek: 9. fejezet). **Nem** következtethetünk belőle általános „algorithmus A minden Snake változaton jobb, mint B” állításra – a

`benchmark_kiertekeles_es_adatvizualizacio_utmutato.md` szerint kerülendő a **túlzott általánosítás**.

- **Futások száma:** a heurisztikák és a tanult stratégiák **különböző N**-nel való mérése **bizonytalanságot** ad az összehasonlításban (lásd [9. fejezet](#) JSON összefoglaló alfejezetét és [10. fejezet](#) záró következtetéseit). Kötelező elem egy szigorúbb dolgozatban: **közös kontrollfutás** (azonos `runs` minden stratégiára).
- **Átlag vs. medián:** csak `score_mean` alapján rangsorolni **félrevezető** lehet; stabil stratégiák jellemzője a **magas medián** és a **szűk eloszlás** (boxplot a `generate_benchmark_plots.py` kimenetén; magyarázat a [5.9 alfejezet](#) és a `Statistics.tsx` metodika szekcióban).
- **Halálprofil olvasata:** sok `starvation` önmagában nem „jó hosszú túlélést” jelent, hanem gyakran **gyenge pontkonverziót** vagy túl szigorú leállítást ([10. fejezet](#), halálprofilok alfejezet). A `self / wall` arányt **együtt** kell értelmezni a pontszámmal.

12.2 Tanuló modellek és tanítási korlátok

- **Hangolás- és erőforrásfüggés:** a DQN / PPO / NEAT eredménye **erősen függ** a jutalomfüggvénytől, a tanítási lépésszámtól / generációszámtól, a megfigyelés dimenziójától és a pályamérettől ([8. fejezet](#)). A mért számok **nem** a módszerek elméleti maximumát adják – csak a **jelenlegi, rögzített beállítások** melletti teljesítményt.
- **Inferencia vs. tanítás:** ha hiányzik a modellfájl, a szolgáltatás **Greedy fallbackre** vált ([7. fejezet](#), tanult stratégiák alfejezet) – benchmarkon a stratégia **neve** alatt **más viselkedés** mérhető, mint „tisza” tanult policy esetén; reprodukálhatósághoz **fájlverziók** rögzítendőek (`benchmark_kiertekeles_es_adatvizualizacio_utmutato.md` §1.2).
- **Általánosítás a tanulókról:** kerülendő a „a tanuló algoritmus alkalmatlan” típusú túlzás; helyette: **„ebben a környezetben és erőforrás-keretben gyengébb, mint a dokumentált heurisztikák”** – összhangban a **webes Statisztika** szövegével.

12.3 Rendszer- és üzemeltetési korlátok

- **Három szolgáltatás:** a teljes élményhez frontend, Node backend és AI szolgáltatás együtt kell; a **Statisztika** és a **MI játék** különösen az **:8000** elérhetőségétől függ ([5.9 alfejezet](#), [4. fejezet](#)).
- **Benchmark fájlok:** az AI szolgáltatás a `benchmarks/results/` útvonalat a fájlrendszerből olvassa – más gépen vagy konténerben **üres** lehet a statisztika, ha a JSON/PNG nincs szinkronban.
- **Vendég eredmények:** az `App.tsx` induláskor `clearScores()` hívása **törölheti** a lokális eredménylistát ([11. fejezet](#), játék vége és pontszám mentés) – demonstrációs / éles használat előtt döntés szükséges.
- **Éles auth:** kötelező `JWT_SECRET` és biztonságos deploy beállítások; a dokumentáció **fejlesztői** alapértelmezéseket mutat.

12.4 Továbbfejlesztési irányok (röviden)

A `docs/ai_docs/szakdolgozat_statisztika_es_elemzes_struktura.md` felveti többek között:

bootstrap konfidencia-intervallum kevés futásnál, **különböző starvation limit** érzékenység, **kisebb pálya** kísérlet, **tanítási költség** (idő / timesteps) összevetése az eredménnyel, **többszempontú rangsor** (median, IQR, first_food, trade-off). Ezek **nem** részei a jelenlegi kötelező megvalósításnak, de **szakdolgozati folytatásként** értelmesek.

12.5 Összefoglaló következtetések

1. A dokumentált benchmark és a **webes Statisztika** együttesen azt mutatja, hogy a **heurisztikák** (különösen az **1 lépéses előretekintés**) a **fix protokollban** erős **baseline**-t adnak; ez **nem** cserélhető fel általános „MI mindig jobb” vagy „heurisztika mindig jobb” állítással.
 2. A **tanuló** modellek a mért konfigurációban **elmaradnak** a top heuristikáktól, de a különbség **nagy részben konfiguráció- és erőforrásfüggő**; a benchmark **értékesen szemlélteti** a paradigmák eltérő viselkedését ugyanazon környezetben.
 3. A rendszer **moduláris**: új stratégia a **STRATEGIES** bővítésével, új metrika vagy plot a benchmark eszközlánc bővítésével jól követhető – a **13. fejezet** rögzíti a fő architekturális döntéseket.
-

13. Architektúra és döntések (jegyzetek)

- **Monorepo**: egy repository, egyszerű verziókezelés és reprodukálhatóság.
 - **Játéklogika független a UI-tól**: **frontend/src/core** tiszta függvények; a Python AI és benchmark ugyanarra az állapotfogalomra épít.
 - **Egységes stratégia-elv**: állapot be → irány ki (frontend **Strategy** interfész; Python **Strategy.next_move**).
 - **Seed és konfiguráció**: localStorage + opcionális fix seed a reprodukálható futásokhoz.
 - **Auth**: JWT Bearer; eredménynél mindig mód és opcionális **ai_strategy**.
 - **Teszt / CI**: a specifikáció említheti a jövőbeli egységtesztet és CI-t – jelen dokumentum csak rögzíti mint továbbfejlesztési irány.
-

14. Irodalomjegyzék

1. **Sutton, R. S. – Barto, A. G.**: *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press, 2018.
 2. **Mnih, V. et al.**: *Human-level control through deep reinforcement learning*. Nature 518, 529–533 (2015).
 3. **Schulman, J. et al.**: *Proximal Policy Optimization Algorithms*. arXiv:1707.06347 (2017).
 4. **Stanley, K. O. – Miikkulainen, R.**: *Evolving Neural Networks through Augmenting Topologies*. Evolutionary Computation 10(2), 99–127 (2002).
 5. **Hart, P. E. – Nilsson, N. J. – Raphael, B.**: *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*. IEEE Trans. Systems Science and Cybernetics, 4(2), 100–107 (1968).
 6. **Russell, S. – Norvig, P.**: *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson, 2020.
-

Dokumentum vége. A részletes kódrészletek és szerkezet összhangban állnak a docs/szakdolgozat_dokumentacio.md megvalósítási naplójával, a terv-jellegű, kizárt források nélkül.